

OreVision — руководство пользователя

Автоматическая классификация руд по панорамным ОМ-изображениям полированных шлифов. Система определяет технологический сорт руды, считает доли фаз по площади и объясняет каждый вердикт картой участков.

Сорт руды	Признак	Цвет на картах
Рядовая	преобладают обычные сростания сульфидов	 зелёный
Труднообогатимая	преобладают тонкие сростания (сульфиды замещены нерудной фазой)	 красный
Оталькованная	доля талька больше порога (по умолчанию 10% площади)	 синий

1. Установка и запуск

Требования: Python 3.11+ (проверено на 3.13), ~2 ГБ на диске. GPU не обязателен — на CPU всё работает, просто медленнее.

```
git clone https://github.com/Fiseldisel/OreVision.git
cd OreVision
python -m venv .venv
.venv\Scripts\activate           # Linux/Mac: source .venv/bin/activate
pip install -r requirements.txt  # версии зафиксированы, конфликтов не будет
```

Для GPU (NVIDIA) вместо torch из requirements поставьте CUDA-сборку: `pip install torch==2.9.1 torchvision==0.24.1 --index-url https://download.pytorch.org/whl/cu126`

Обученная базовая модель (`models/base.pt`) уже лежит в репозитории — обучать ничего не нужно.

Запуск:

```
streamlit run app.py
```

Приложение откроется в браузере на `http://localhost:8501`. Остановить — `Ctrl+C` в терминале.

2. Вкладка « Анализ изображений»

Как загрузить изображение

Три способа, любой на выбор:

1. **Кнопки «Пример: ...»** — встроенные образцы каждого сорта и фрагмент панорамы; демо в один клик, свои файлы не нужны.
2. **Перетащить файл** в область загрузки (TIFF/PNG/JPEG/BMP, до ~1 ГБ).
3. **Указать путь к файлу или папке на диске** — самый быстрый способ для больших панорам: файл читается напрямую, без загрузки через браузер.

Затем — кнопка «► **Анализировать**». Обычное фото занимает меньше секунды на GPU (секунды на CPU), панорама в 150 Мпикс — около 10 секунд на GPU (минуты на CPU).

Режим анализа (сайдбар)

- **auto** (по умолчанию): изображения крупнее 30 Мпикс анализируются как панорамы, остальные — как фото. Менять обычно не нужно.
- **photo** — изображение целиком приводится к масштабу обучения и оценивается по сетке перекрывающихся участков; вердикт — по средним вероятностям.
- **panorama** — разбиение на тайлы 1024×1024 в нативном разрешении; вердикт — по экспертному правилу через доли площади.

Как читать результат

- **Баннер вердикта** — итоговый сорт руды. Цвет баннера соответствует классу.
- **⚠ Пограничный случай** — появляется, когда доли конкурирующих классов близки или доля талька рядом с порогом: вердикту не стоит доверять слепо, рекомендована проверка эксперта.
- **Полоса состава** — доли площади трёх классов одним взглядом; чёрная риска — порог талька (отсчитывается от правого края).
- **Текстовое заключение** — готовая формулировка для отчёта лаборатории.
- Вкладки под заключением:
- **Карта классов** — цветовая маска поверх шлифа. Наведите курсор на любой участок: всплывёт подсказка с вероятностями всех классов и уверенностью. Колесо мыши — зум, перетаскивание — панорамирование.
- **Карта уверенности** — жёлтые участки модель классифицирует уверенно, тёмно-синие — спорные, на них стоит посмотреть глазами.
- **Исходное** — изображение без маски.
- **Спорные участки** — топ-6 участков с наименьшей уверенностью, с координатами (ряд/столбец). Это очередь на ручную проверку и вход в режим дообучения (см. раздел 4).
- **Таблица метрик** — доли площади по классам, суммарные сульфиды, доля талька, средние вероятности, число тайлов, время анализа.

Настройки в сайдбаре

- **Порог доли талька** — экспертное правило «тальк > порога → оталькованная». При изменении вердикт уже проанализированных панорам пересчитывается мгновенно, без повторного инференса.
- **Прозрачность маски** — насколько плотно карта классов закрашивает шлиф.
- **Чекпойнт** — выбор версии модели, если в `models/` их несколько.

Экспорт

Под таблицей метрик: **Оверлей (JPG)**, **Карта уверенности (JPG)**, **Отчёт (PDF)** — одностраничный отчёт с картами, метриками и заключением, **Метрики (CSV)**, и  **всё одним ZIP-архивом**.

3. Вкладка « **Пакетная обработка**»

Обработка папки целиком без участия пользователя.

1. Укажите **папку с изображениями** (поддерживаются те же форматы).

2. При желании поменяйте папку результатов (по умолчанию `results/run_<data>`).
3. Отметьте, что генерировать: PDF-отчёты, оверлеи.
4. «▶ Запустить пакетную обработку» — по каждому файлу виден прогресс.

Результат: сводная таблица (цветные вердикты, доли-прогрессбары, флаг  у пограничных случаев) и папка с артефактами:

```
results/run_20260704_120000/
├─ <имя файла>/overlay.jpg      # карта классов
├─ <имя файла>/confidence.jpg   # карта уверенности
├─ <имя файла>/report.pdf       # отчёт
├─ summary.csv                 # сводка по всем файлам
└─ run_params.json             # конфиг, версия модели, список файлов —
                                # для воспроизводимости анализа
```

То же самое без интерфейса (для встраивания в конвейер лаборатории):

```
python -m orevision.cli --input "D:/шлифы/партия_07" --pdf
python -m orevision.cli --input "D:/шлифы/панорама.tif" --pdf # один файл
```

4. Вкладка « Дообучение» — режим экспертной проверки

Модель учится у геолога. Рабочий цикл:

Шаг 1 — исправить ошибки модели.

- Ошибка на участке панорамы: вкладка результата «Спорные участки» → под участком выбрать верный класс → « В набор дообучения». Участок сохраняется в нативном разрешении.
- Ошибка на целом снимке: развернуть « Экспертная коррекция вердикта» под кнопками экспорта → выбрать верный класс → «Сохранить пример».

Шаг 2 — посмотреть, что накопилось (и убрать лишнее). На вкладке « Дообучение» — счётчики примеров по классам и журнал: какой участок, из какого файла, что предсказывала модель и какая версия модели ошибалась. Ошибочно добавленные примеры удаляются там же: отметьте строки в журнале → « Удалить выбранные»; кнопка « Очистить весь набор» (с подтверждением) сбрасывает набор целиком.

Шаг 3 — дообучить модель.

-  **Быстрое дообучение** — деликатная дошлифовка активной модели с учётом ваших исправлений (несколько эпох, малый шаг). Рекомендуется в повседневной работе. Результат — новый файл `..._дообученная.pt`.
-  **Полное переобучение** — обучение с нуля на всех данных плюс исправления. Для случаев, когда исправлений накопилось много. Результат — `..._переобученная.pt`.

Важно про версии моделей. Обучение **не перезаписывает** базовую модель `base.pt` — каждый запуск создаёт **новый** чекпойнт, который появляется в списке «Чекпойнт» в сайдбаре и сразу выбирается активным. Имена понятные: `base.pt` (базовая), `base_дообученная.pt`, `base_дообученная_2.pt`, `base_переобученная.pt` и т.д. Качество (val macro-F1) в сайдбаре и на вкладке «О модели» всегда соответствует **выбранной** модели.

Шаг 4 — сравнить и выбрать. Переключайтесь между версиями в списке «Чекпойнт» — это и есть «откат»: чтобы вернуться к базовой модели, просто выберите `base.pt`. Ненужную обученную версию можно удалить кнопкой « Удалить эту версию» (базовую удалить нельзя).

Честность метрик: исправленные примеры попадают **только в обучающую выборку**, валидационная не меняется — цифры качества нельзя «накрутить» фидбэком.

Важно: запуск дообучения требует обучающего датасета на этой же машине — исправления подмешиваются к нему, на нескольких примерах в отрыве от датасета модель дообучить нельзя. Если датасета нет, кнопки обучения неактивны, а на вкладке появляется поле «**Указать путь к датасету**» — введите путь, он сохранится в `config.local.yaml` (машинная настройка, в git не попадает). Формат датасета — см. раздел 6. Сбор исправлений работает и без датасета: они копятя в `data/feedback/`, папку можно перенести на машину с датасетом.

5. Вкладка « О модели и данных»

Качество активной модели на валидации: macro-F1, ROC-AUC, F1 по классам, разбивка по частям датасета (разные условия съёмки), матрица ошибок. Рядом — краткое описание, как работает система. Текущая модель: EfficientNetV2-S (transfer learning с ImageNet), вход 384×384, валидация на 220 изображениях с групповым сплитом (фото одной пробы никогда не разделяются между обучением и проверкой): **macro-F1 ≈ 0,93, ROC-AUC ≈ 0,97**.

6. Обучающий датасет и переобучение (для инженера)

Формат датасета

Система принимает датасет в одном из двух видов (распознаётся автоматически):

1. **Датасет хакатона** — папка «Задача 3. Скажи мне, кто твой шлиф» со своей исходной структурой (Фото руд по сортам. ч1/ч2 и т.д.). Достаточно указать путь к её корню.
2. **Свой набор** — папка с тремя подпапками по классам; внутри — изображения (JPG/JPEG/PNG/TIFF/BMP). Имена подпапок (регистр не важен):

Класс	Допустимые имена папки
Рядовая	<code>ordinary</code> , <code>рядовая</code> , <code>рядовые</code> (и варианты вида «Рядовые руды»)
Труднообогатимая	<code>hard</code> , <code>труднообогатимая</code> , <code>труднообогатимые</code> , <code>тонкие</code>
Оталькованная	<code>talc</code> , <code>оталькованная</code> , <code>оталькованные</code> , <code>тальк</code>

Нужны все три класса, рекомендуем от 30 фото на класс. Пример:

`D:\мой_датасет\` | `— рядовые\ *.jpg` | `— тонкие\ *.jpg` | `— оталькованные\ *.jpg`

Куда прописать путь

Любой из способов:

- в интерфейсе: вкладка « Дообучение» → « Указать путь к датасету» (появляется, когда датасет не найден) — путь проверится и сохранится;
- вручную: создать рядом с `config.yaml` файл `config.local.yaml`:

```
yaml data: root: "D:/мой_датасет"
```

`config.local.yaml` — локальная настройка машины, в git не попадает; сам `config.yaml` править не нужно.

Переобучение

```
python -m orevision.tools.build_manifest --cache # аудит: дубликаты, конфликты, сплит
python -m orevision.train                       # обучение (~8 мин на GPU)
```

Аудит сам найдёт точные и перцептивные дубликаты, исключит файлы с конфликтующей разметкой и построит сплит без утечки по пробам. Все артефакты обучения (метрики, матрица ошибок, кривые, лог) сохраняются в `models/`.

Ключевые настройки — в `config.yaml`: порог талька, размер тайла панорамы, порог «панорамности» (30 Мпикс), параметры обучения.

7. Частые вопросы и проблемы

Нет подсказок при наведении на карту классов. Не установлен plotly: `pip install plotly` и перезапустите приложение (в свежем `requirements.txt` он уже есть).

«Нет чекпойнтов в models» — отсутствует `models/base.pt`. Заберите его из репозитория (входит в поставку) или обучите модель (раздел 6).

Порт 8501 занят — `streamlit run app.py --server.port 8502`.

Файл больше 1 ГБ не загружается через браузер — используйте поле «путь к файлу на диске»: оно без ограничений по размеру.

Вердикт «Не определено» — на изображении не нашлось пригодных для анализа участков (практически однотонное: фон, эпоксидка, пересвет). Проверьте образец.

Долго на CPU — панорама в сотни Мпикс на CPU считается минутами; это нормально. Требование T3 (панорама 10 000×10 000 за ≤5 минут) выполняется и на CPU; GPU ускоряет анализ до секунд.

Кнопки дообучения неактивны / «Дообучение на этой машине недоступно» — не найден обучающий датасет. Укажите путь к нему прямо на вкладке «Дообучение» (поле «📁 Указать путь к датасету») — подойдёт датасет хакатона или свой набор из трёх папок-классов (формат — раздел 6). Либо соберите исправления здесь ([data/feedback/](#)) и дообучите на машине, где датасет есть.

Как вернуть базовую модель после экспериментов с дообучением — в сайдбаре в списке «Чекпойнт» выберите `base.pt`. Базовая модель обучением не перезаписывается, поэтому всегда на месте.